Concurrent Object Detection Pipeline

Prepared by:
Anthony Al Hassrouty
Vanessa Mghames
Perla Bou Chebel

Presented to : Dr Mohamad Aoude

Project Overview

What it does

Detects objects in images using a 3-stage concurrent Java pipeline connected to a Python YOLOv8 AI service over HTTP.

Topic: Machine Learning Pipeline (Java calling a Python ML service)

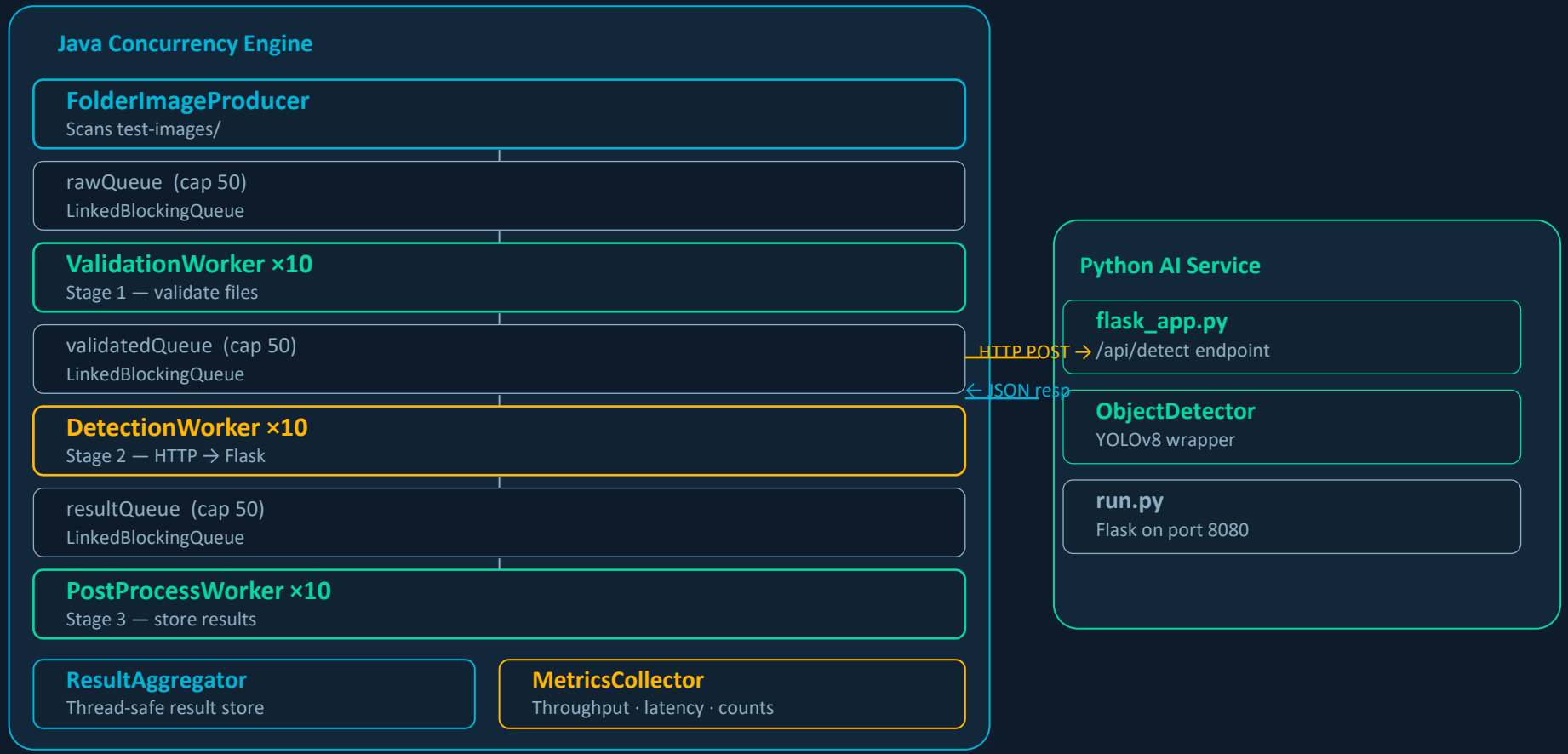
Real-world need: sequential processing is 10× slower. Each image requires ~1,400 ms neural network inference.

30 Concurrent threads

3×50 Bounded queues (cap 50 each)

7.84× Speedup vs sequential

System Architecture



Concurrency Design

01

Fixed thread pool

Executors.newFixedThreadPool(30) — 10 per stage. Stage 2 is I/O-bound (waiting for Flask), so threads >> CPU cores is correct.

02

Bounded queues

3 × LinkedBlockingQueue(50). When downstream is slow, upstream put() blocks — backpressure, not OOM. Unbounded queues explicitly rejected.

03

Poison pill shutdown

Producer injects 10 poison pills per queue. Every worker receives exactly one, breaks its loop, and forwards pills to the next stage.

04

Thread-safe shared state

AtomicInteger, AtomicLong, ConcurrentHashMap, synchronized addResult(). Python side: threading.Lock. No raw int++ on shared fields anywhere.

No unbounded resources — fixed pool (30 threads) + 3 capped queues + no newCachedThreadPool()

Distributed Boundary — Java ↔ Python over HTTP

Java — DetectionWorker

- HTTP POST /api/detect with image path
- connectTimeout = 5 seconds
- readTimeout = 5 minutes
- Automatic retry on IOException
- Per-image log: OK / Retrying / ERROR
- CountDownLatch awaits all workers

HTTP POST →

← JSON resp

Python — Flask / YOLOv8

- Flask threaded=True (10 concurrent requests)
- POST /api/detect receives image path
- YOLOv8 runs neural net inference
- Returns JSON list of detected objects
- GET /api/health — liveness check
- threading.Lock protects shared counters

Two separate processes. Real network boundary with serialization, timeout, logging, and failure handling.

Failure Injection & Recovery

× Flask not running

Inject: Start Java without starting Python

Expected: All 34 images retry once → all logged ERROR after retry

Result: Failed: 34 / Completed: 0 — pipeline exits cleanly, no hang

Flask killed mid-run

Inject: Start Flask, start Java, kill Flask after ~5 images

Expected: Remaining images retry once, then fail gracefully

Result: Partial completed + partial failed — no crash, no hang

? Invalid file in folder

Inject: Add a .txt or .pdf into test-images/

Expected: [Validation] SKIPPED (invalid): ... logged at Stage 1

Result: Rejected: 1 — pipeline continues processing all other images

⏸ Queue backpressure

Inject: Reduce queue capacity from 50 to 2 in MainApp

Expected: Producer blocks on put() when queue fills up

Result: Intentional backpressure — not a deadlock, system recovers

Load Test — Normal Load (34 images)

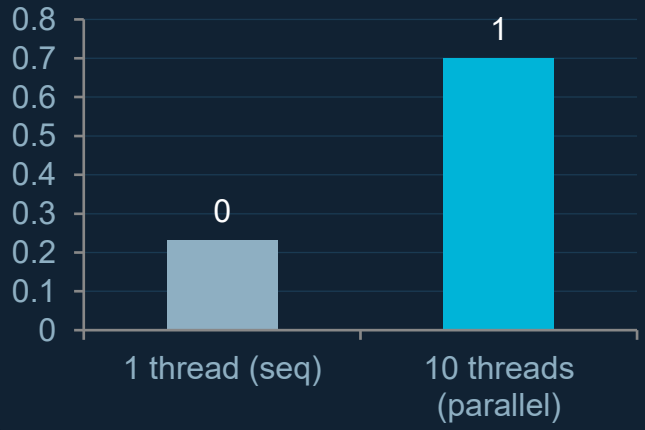
Metric	Value
Pipeline duration	~48,000 ms
Throughput	0.70 img/s
Average latency per image	1,382 ms
Max latency per image	4,201 ms
Completed	34
Failed	0
Retried	0
Queue depths at shutdown	0 / 0 / 0

Benchmark

Sequential (1 thread)
400 ms pre-processing

Parallel (10 threads)
51 ms pre-processing

7.84× speedup



Stress Test — Overload (102 images)

Metric	Value
Pipeline duration	509,986 ms
Throughput	0.20 img/s
Average latency	47,710 ms
Max latency	52,864 ms
Completed	102
Failed	0
Retried	9
Queue depths at shutdown	0 / 0 / 0

102/102

Images completed — 0 lost

9

Retries absorbed transparently

0/0/0

Queue depths at shutdown

102 > queue cap 50 → backpressure activated. Producer blocked on put(). Controlled pause, not a crash.

Architecture Decision — Bounded Queue Capacity 50

Our choice

Bounded queue (cap 50)

- Memory capped at 50 items per queue — no OOM
- Backpressure: producer pauses when downstream stalls
- Required by assignment (unbounded = instant fail)
- Overload is observable: queue depth metric shows pressure
- Proved correct: 102 images, 0 lost, 0 failures
- Deadlock-safe: capacity (50) always > poison pills (10)

Rejected

Unbounded queue

- Queue grows until JVM runs out of heap → OOM crash
- No backpressure — producer floods memory instantly
- Hides bottleneck — Stage 2 slowness is invisible
- Explicitly banned by the assignment specification
- Depth metric becomes meaningless (always growing)
- Failure under load is silent, not measurable

"Bounded queues turned a potential OOM crash into a controlled, measurable pause — with zero data loss."

Concurrency Correctness Scorecard

Question	Mechanism used
Thread-safe?	ConcurrentHashMap, AtomicInteger, synchronized, threading.Lock (Python)
Visibility guaranteed?	AtomicInteger/Long use volatile internally; volatile pipelineStartMs in MetricsCollector
Deadlock-free?	Single lock per method, no nested locking, no circular wait possible
Liveness guaranteed?	Poison pill guarantees every worker terminates; CountDownLatch in MainApp; awaitTermination(30s)
Bounded resources?	newFixedThreadPool(30) — 3 queues LinkedBlockingQueue(50) — no newCachedThreadPool()
Failure recovery?	IOException caught in DetectionWorker → retry once → recordFailed() → pipeline continues

102 images processed under overload · 0 failures · 0 items lost · All queue depths 0 at shutdown